

SQL Cheat Sheet – PostgreSQL vs ClickHouse

Kontekst

Ściąga jest przygotowana jako pomoc do laboratoriów z porównania PostgreSQL i ClickHouse na tabeli events.

Przykładowe kolumny tabeli events:

- user_id
- session_id
- event_time
- event_type
- country
- device
- price
- quantity

W zależności od bazy tabela może być dostępna jako:

- PostgreSQL: `public.events`
- ClickHouse: `ds_lab.events`

1. Agregacje podstawowe

Operacja	PostgreSQL	ClickHouse	Różnica
Liczba rekordów	COUNT(*)	count() albo COUNT(*)	ClickHouse preferuje count()
Suma	SUM(price)	sum(price)	Identyczne
Średnia	AVG(price)	avg(price)	Identyczne
Minimum	MIN(price)	min(price)	Identyczne
Maksimum	MAX(price)	max(price)	Identyczne

Przykład: łączny przychód z zakupów.

```
SELECT SUM(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase';
```

2. Liczenie unikalnych wartości

Cel	PostgreSQL	ClickHouse	Uwagi
Dokładna liczba unikalnych	COUNT(DISTINCT user_id)	uniqExact(user_id)	uniqExact daje dokładny wynik
Szybkie przybliżenie	Brak odpowiednika	uniq(user_id)	Wynik przybliżony, ale szybszy

PostgreSQL:

```
SELECT COUNT(DISTINCT user_id) AS users_count  
FROM events;
```

ClickHouse:

```
SELECT uniqExact(user_id) AS users_count  
FROM events;
```

3. Agregacje warunkowe

To jedna z najważniejszych różnic między klasycznym SQL a idiomatycznym ClickHouse.

Cel	PostgreSQL / klasyczny SQL	ClickHouse
Liczba zakupów	SUM(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END)	countIf(event_type = 'purchase')
Suma przychodu z zakupów	SUM(CASE WHEN event_type = 'purchase' THEN price * quantity ELSE 0 END)	sumIf(price * quantity, event_type = 'purchase')
Czy sesja miała zakup	MAX(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END)	countIf(event_type = 'purchase') > 0

PostgreSQL:

```
SELECT
    session_id,
    MAX(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END) AS has_purchase
FROM events
GROUP BY session_id;
```

ClickHouse:

```
SELECT
    session_id,
    countIf(event_type = 'purchase') > 0 AS has_purchase
FROM events
GROUP BY session_id;
```

4. Praca z datami

Cel	PostgreSQL	ClickHouse
Data z daty-czasu	DATE(event_time)	toDate(event_time)
Początek miesiąca	DATE_TRUNC('month', event_time)	toStartOfMonth(event_time)
Początek tygodnia	DATE_TRUNC('week', event_time)	toStartOfWeek(event_time)
Różnica w dniach	DATE_PART('day', t1 - t2)	dateDiff('day', t2, t1)
Różnica w minutach	EXTRACT(EPOCH FROM (t1 - t2)) / 60	dateDiff('minute', t2, t1)

PostgreSQL: liczba zdarzeń dziennie.

```
SELECT
    DATE(event_time) AS day,
    COUNT(*) AS event_count
FROM events
GROUP BY DATE(event_time)
ORDER BY day;
```

ClickHouse: liczba zdarzeń dziennie.

```
SELECT
    toDate(event_time) AS day,
    count() AS event_count
FROM events
GROUP BY day
ORDER BY day;
```

5. CTE – WITH

CTE (Common Table Expression) pozwala zapisać podzapytanie i wykorzystać je dalej. Składnia jest identyczna w obu bazach.

```
WITH sessions AS (  
  SELECT  
    session_id,  
    MAX(CASE WHEN event_type = 'purchase' THEN 1 ELSE 0 END) AS has_purchase  
  FROM events  
  GROUP BY session_id  
)  
SELECT  
  COUNT(*) AS sessions_count,  
  SUM(has_purchase) AS purchase_sessions  
FROM sessions;
```

6. Funkcje okna

Funkcje okna liczą wartości w kontekście innych rekordów, bez zwiżania danych tak jak w GROUP BY.

Cel	PostgreSQL	ClickHouse
Ranking	RANK() OVER (...)	RANK() OVER (...)
Gęsty ranking	DENSE_RANK() OVER (...)	DENSE_RANK() OVER (...)
Poprzednia wartość	LAG(x) OVER (...)	lagInFrame(x) OVER (...)
Suma narastająca	SUM(x) OVER (ORDER BY day)	sum(x) OVER (ORDER BY day ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

Przykład: ranking krajów według przychodu.

```
SELECT
  country,
  SUM(price * quantity) AS revenue,
  RANK() OVER (ORDER BY SUM(price * quantity) DESC) AS rank_no
FROM events
WHERE event_type = 'purchase'
GROUP BY country
ORDER BY rank_no;
```

Wskazówka: suma narastająca

Zbuduj w dwóch krokach — najpierw CTE z agregacją dzienną, potem funkcja okna:

```
WITH daily AS (
  SELECT ... AS day, SUM(...) AS daily_revenue
  FROM events
  WHERE event_type = 'purchase'
  GROUP BY day
)
SELECT day, daily_revenue,
  SUM(daily_revenue) OVER (ORDER BY day) AS cumulative
FROM daily;
```

W ClickHouse dodaj ramkę: ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW.

7. Wzorce składniowe: lejek konwersji

Lejek mierzy, ile sesji przechodzi przez kolejne etapy: view → cart → purchase.

Kluczowe wzorce

Flaga „czy sesja miała zdarzenie X”:

```
-- PostgreSQL
MAX(CASE WHEN event_type = 'view' THEN 1 ELSE 0 END) AS has_view

-- ClickHouse
countIf(event_type = 'view') > 0 AS has_view
```

Bezpieczne dzielenie (unikanie dzielenia przez zero):

```
-- PostgreSQL
1.0 * SUM(has_cart) / NULLIF(SUM(has_view), 0)

-- ClickHouse
countIf(has_cart) / nullIf(countIf(has_view), 0)
```

Schemat budowy

1. CTE z flagami per sesja (GROUP BY session_id)
2. Agregacja flag do poziomu całego zbioru (SUM lub countIf)
3. Obliczenie wskaźników przejścia (dzielenie z NULLIF)

Pełne zapytanie zbuduj samodzielnie, łącząc powyższe wzorce z CTE (sekcja 5).

8. Wzorce składniowe: analiza sesji

Kluczowe wzorce

Czas trwania sesji w minutach:

```
-- PostgreSQL
EXTRACT(EPOCH FROM (MAX(event_time) - MIN(event_time))) / 60

-- ClickHouse
dateDiff('minute', min(event_time), max(event_time))
```

Liczba unikalnych typów zdarzeń:

```
-- PostgreSQL
COUNT(DISTINCT event_type)

-- ClickHouse
uniqExact(event_type)
```

Schemat budowy

1. CTE z metrykami per sesja (GROUP BY session_id): event_cnt, unique_types, duration_min, has_purchase
2. Segmentacja sesji w zewnętrznym zapytaniu (CASE WHEN na duration_min)
3. Agregacja per segment (COUNT, AVG, SUM)

9. Wzorce składniowe: segmentacja RFM

Kluczowe wzorce

Data referencyjna jako CTE:

```
WITH ref AS (  
    SELECT MAX(event_time) AS ref_time FROM events  
)
```

Obliczanie recency:

```
-- PostgreSQL  
DATE_PART('day', ref.ref_time - p.last_purchase_time)  
  
-- ClickHouse  
dateDiff('day', p.last_purchase_time, ref.ref_time)
```

Łączenie z datą referencyjną:

```
FROM purchases p CROSS JOIN ref
```

Schemat budowy

1. CTE ref z datą referencyjną
2. CTE purchases z frequency, monetary, last_purchase_time per user_id
3. SELECT łączący oba CTE przez CROSS JOIN i obliczający recency
4. Segmentacja na podstawie wybranych progów (CASE WHEN)

Pełne zapytanie zbuduj samodzielnie, łącząc powyższe wzorce.

10. EXPLAIN i analiza planu

Cel	PostgreSQL	ClickHouse
Pokazanie planu	EXPLAIN SELECT ...	EXPLAIN SELECT ...
Plan z realnym czasem	EXPLAIN ANALYZE SELECT ...	Analizujemy przez czas wykonania i EXPLAIN

Przykład PostgreSQL:

```
EXPLAIN ANALYZE
SELECT country, SUM(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase'
GROUP BY country;
```

Przykład ClickHouse:

```
EXPLAIN
SELECT country, sum(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase'
GROUP BY country;
```

11. Cache zapytań w ClickHouse

Przy benchmarku trzeba uważać na cache wyników.

```
SET use_query_cache = 0;
```

Jeżeli cache jest włączony, kolejne wykonanie identycznego zapytania może być sztucznie bardzo szybkie, bo wynik zostanie pobrany z pamięci podręcznej, a nie policzony od nowa.

12. Najważniejsze różnice do zapamiętania

Obszar	PostgreSQL / klasyczny SQL	ClickHouse
Główne zastosowanie	System transakcyjny, relacyjny	Analityka dużych danych, OLAP
Agregacje	Standardowe COUNT, SUM, AVG	Bardzo szybkie agregacje kolumnowe
Warunkowe liczenie	CASE WHEN	countIf, sumIf
Unikalne wartości	COUNT(DISTINCT ...)	uniqExact, uniq
Daty	DATE, DATE_TRUNC, DATE_PART	toDate, toStartOfMonth, dateDiff
Benchmark	Stabilny, zwykle wolniejszy na dużych agregacjach	Bardzo szybki, ale trzeba uważać na cache
Czytelność zapytań analitycznych	Często dłuższe konstrukcje	Krótsze funkcje analityczne

13. Miniściągą funkcji ClickHouse

Funkcja	Co robi	Przykład
<code>count()</code>	Liczy rekordy	<code>count()</code>
<code>countIf(condition)</code>	Liczy rekordy spełniające warunek	<code>countIf(event_type = 'purchase')</code>
<code>sumIf(value, condition)</code>	Sumuje warunkowo	<code>sumIf(price * quantity, event_type = 'purchase')</code>
<code>uniqExact(x)</code>	Dokładnie liczy unikalne	<code>uniqExact(user_id)</code>
<code>uniq(x)</code>	Szybko, przybliżenie	<code>uniq(user_id)</code>
<code>toDate(x)</code>	Data z daty-czasu	<code>toDate(event_time)</code>
<code>toStartOfWeek(x)</code>	Początek tygodnia	<code>toStartOfWeek(event_time)</code>
<code>toStartOfMonth(x)</code>	Początek miesiąca	<code>toStartOfMonth(event_time)</code>
<code>dateDiff(unit, start, end)</code>	Różnica dat	<code>dateDiff('day', last_purchase, ref_time)</code>

14. Podsumowanie

Najważniejsze funkcje ClickHouse:

```
countIf(...)  
sumIf(...)  
uniqExact(...)  
toDate(...)  
toStartOfMonth(...)  
dateDiff(...)
```

Najważniejsza różnica praktyczna:

- w PostgreSQL częściej piszemy klasyczne konstrukcje SQL, np. CASE WHEN,
- w ClickHouse warto używać dedykowanych funkcji analitycznych: countIf, sumIf, uniqExact - są krótsze, czytelniejsze i naturalne dla tej bazy.